



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET

Katedra za računarstvo



Istraživanje performansi različitih modela dubokog učenja za sentiment analizu korisničkih recenzija

Seminarski rad

Predmet: Duboko učenje

Mentor:

Aleksandar Milosavljević

Studenti:

Nastasija Stanković 1622
Katarina Stanojković 1773

Sadržaj

1. Uvod.....	3
2. Analiza sentimenta.....	4
2.1. Teorijske osnove analize sentimenta.....	4
2.2. Primena analize sentimenta.....	4
2.3. Tehnike obrade tekstualnih podataka.....	5
2.3.1. Preprocesiranje podataka.....	5
2.3.2. Priprema podataka za treniranje.....	7
3. Praktični deo rada.....	12
3.1. Učitavanje biblioteka.....	12
3.2. Skup podataka.....	12
3.3. Preprocesiranje podataka.....	14
3.4. Eksplorativna analiza podataka.....	15
3.4.1. Vizualizacija raspodele sentimenta.....	15
3.4.2. Vizualizacija najučestalijih reči pomoću WordCloud.....	16
3.4.3. N-gram vizuelizacija.....	17
3.5. Priprema podataka za treniranje modela.....	19
3.5.1. Vektorizacija tekstova korišćenjem 1-gram reprezentacija.....	20
3.5.2. Vektorizacija tekstova korišćenjem 3-gram reprezentacija.....	22
3.6. Treniranje i evaluacija modela.....	23
3.6.1. Treniranje i evaluacija modela mašinskog učenja.....	24
3.6.2. Treniranje i evaluacija modela zasnovanih na BERT arhitekturi.....	27
3.6.3. Treniranje i evaluacija veštačkih neuronskih mreža.....	28
4. Zaključak.....	32
5. Reference.....	33

1. Uvod

U savremenom dobu gde se veštačka inteligencija širom primenjuje u različitim oblastima softverskog razvoja, ključno je razumeti njen način funkcionisanja i prepoznati potencijalne mogućnosti za njenu primenu. Da bismo identifikovali te mogućnosti, prvo je bitno steći uvid u različite tehnologije, savladati brojne koncepte koji su razvijeni u poslednje vreme, a takođe i razumeti kako primeniti određene tehnologije u konkretnim scenarijima. Kroz ova razumevanja, moguće je kreirati sistem veštačke inteligencije sposoban za rešavanje različitih problema.

Sistemi za detekciju, razumevanje i kontekstualizaciju tekstualnih zapisa predstavljaju široku oblast koja se veoma brzo razvija početkom treće decenije 21. veka. Ubrzani razvoj i pomama za ovakvim sistemima kreće krajem 2022. godine sa pojavom do sada neviđenog sistema za obradu tekstova pod nazivom eng. ChatGPT. Široke korisničke mase koje su se uz pomoć ovog sistema upoznale sa mogućnosti koju nude sistemi za obradu tekstova, dodatno su ubrzale dalji razvoj i napredak u obradi prirodnih jezika stvarajući nove slučajeve korišćenja ovakve tehnologije.

Analiza sentimenata predstavlja jedan od osnovnih slučajeva korišćenja veštačke inteligencije. Koriste se procesi klasičnog mašinskog učenja, veštačkih neuronskih mreža i procesi obrade prirodnih jezika kako bi se kreirala jedna simbioza različitih oblasti sa ciljem rešavanja specifičnih problema poput analize sentimentalnih osobina tekstualnih zapisa.

Dalji tekst ovog rada je podeljen na teorijski i praktični deo. U teorijskom delu rada su obrađene tehnike obrade tekstualnih podataka, dok su u praktičnom delu rada prikazani rezultati ovih obrada.

Teorijski deo rada obuhvata osnovnu teorijsku pozadinu iza tehnika obrade tekstualnih podataka, dok praktični deo rada obuhvata primenu navedenih tehnika nad specifičnim skupom podataka sa primenom klasifikacionih algoritama mašinskog učenja, kao i uz pomoć pretreniranih jezičkih modela.

2. Analiza sentimenta

Analiza sentimenta predstavlja slučaj korišćenja primene klasifikacije. Klasifikacija teksta se bavi proučavanjem označenih skupova podataka pri čemu se skup podataka sastoji od tekstualnog zapisa i kategoričkog atributa koji predstavlja kategoriju pripadnosti iz skupa konačnih kategorija. Analiza sentimenta je slučaj u kojem tekstualni podaci nose sentimentalnu kontekstualnu vrednost na osnovu kojih je moguće odrediti sentiment datog podatka i kategorisati ga u neku od mogućih sentimentalno-kvalitativnih kategorija.

2.1. Teorijske osnove analize sentimenta

Koncept analize sentimenta kao specifičnog slučaja obrade tekstualnih podataka predstavlja proces utvrđivanja kvalitativnih svojstava emocionalnog tonusa tekstualnih zapisa kako bi se iz datog skupa podataka izvele određene zakonitosti koje važe između kontekstualnih reprezentacija posmatranih emocionalnih zapisa i kategoričkog atributa koji predstavlja sentimentalni pokazatelj pripadnosti određenom skupu.

Cilj ovog procesa jeste utvrđivanje emocionalnog konteksta datih tekstova i težinu koju reči prirodnog jezika imaju u odnosu na stanje sentimenta koji opisuju. Stavovi tj. kontekst koji se prenosi kroz tekst se obično označavaju u nekoliko kategorija poput:

- Pozitivno
- Negativno
- Neutralno

2.2. Primena analize sentimenta

Proces utvrđivanja sentimenta u tekstu sa sobom donosi nove slučajeve primene ovakvog tipa tehnologija u različitim sferama. Primena analize sentimenta ima najvećih benefita u sferama koje se bave obradom i analizom mišljenja korisnika, ocenu određenih entiteta (brendova, proizvoda, itd.), nivoima oduševljenja ili razočarenja posmatranog korisnika i sl. Ovakvi slučajevi korišćenja mogu dodatno doprineti analizi određenih pojava jer do sada nije bilo moguće automatizovati analizu i procesiranje tekstova na ovaj način.

Ovakav automatizovan način obrade velikih tekstualnih skupova podataka sa sobom donosi ubrzani način obrade i analize različitih tekstova.

Konačni rezultati utvrđivanja konteksta tekstova imaju za cilj prezentovanje jasnije slike o stečenim mišljenjima različitih korisnika nad datim objektima posmatranja pri čemu se u analizu mogu uključiti i dodatni parametri koji mogu dodatno kategorizovati tipove ličnosti korisnika.

2.3. Tehnike obrade tekstualnih podataka

Tehnike obrade prirodnog jezika predstavljaju tehnike kojima se obrađuju podaci koji se u izvornom obliku nalaze u tekstualnom formatu. Ovakvim tehnikama se vrši obrada i priprema podataka za dalje korake u razvoju procesa analize sentimenata poput primene matematičkih metoda mašinskog učenja ili treniranja modela.

Obzirom da nad podacima koji se izvorno nalaze u tekstualnom formatu nije moguće primeniti spomenute dalje korake, takve podatke je najpre neophodno pretvoriti u pogodan format koji se može obrađivati na hardveru koji se koristi u mašinskom učenju. Pogodan format jeste vektorski/tenzorski prostor u kojem je neophodno predstaviti tekstualni kontekst posmatranog skupa podataka.

Obrađeni tekstualni podaci postaju vektori koji se u vektorskom/tenzorskom prostoru predstavljaju kao nizovi brojeva određene dužine. Vrednosti koje ovi vektori nose sa sobom treba da na visoko-kvalitativan način opišu dati tekstualni podatak pri čemu treba održati svojstva tekstova poput redosleda reči u rečenici, kontekstualne zavisnosti između reči, dužina tekstova, kontekstualna sličnosti i sl.

2.3.1. Preprocesiranje podataka

Preprocesiranje podataka se bavi osnovnim čišćenjem ulaznih tekstualnih podataka i podrazumeva neke od sledeći tehnika:

- Čišćenje teksta
- Normalizacija
- Tokenizacija
- Lematizacija i stemovanje
- Uklanjanje stop-reči

Čišćenje teksta

Čišćenje teksta je prvi korak u preprocesiranju tekstualnih podataka i podrazumeva uklanjanje specijalnih karaktera iz teksta poput HTML tagova, znakova interpunkcije, brojeva i ostalih ne-tekstualnih karaktera. Na ovaj način se vrši eliminacija suvišnih simbola iz teksta i krera jednostavan i čistiji tekst.

Normalizacija

Normalizacija predstavlja korak u preprocesiranju tekstualnih podataka pri čemu se vrši konverzija teksta u jedinstveni tj. standardni format za sve karaktere teksta. Osnovni proces u normalizaciji jeste case konverzija pri čemu se tekst pretvara u lower ili upper case.

Tokenizacija

Tokenizacija predstavlja korak u preprocesiranju tekstualnih podataka pri čemu se ulazna rečenica deli na listu kraćih tekstualnih jedinica koje mogu biti pojedinačne reči ili delovi reči, čak i pojedinačna slova. Tokeni predstavljaju smislenu celinu koja predstavlja zajednički kontekst određenih delova reči, tj. celine tokenizovane rečenice. U najvećem broju slučajeva se tokenizacija vrši podelom ulazne rečenice na listu reči. Ovakva lista je pogodna za dalje korake u preprocesiranju podataka.

Lematizacija i stemovanje

Ove tehnike predstavljaju tehnike obrade teksta koje se koriste za konverziju i skraćivanje reči na njihov osnovni oblik ili koren reči. Tehnike se primenjuju na identičan način pri čemu se primenjuju različite transformacije.

Stemovanje predstavlja jednostavniji proces obrade reči pri čemu se posmatrana reč svodi na koren reči izveden iz posmatrane. Dodatna bitna karakteristika jeste i vreme izvršavanja samog procesa obrade podataka. Takođe, stemovanje se obavlja nad svakom rečju pojedinačno bez dodatnog poznavanja konteksta ostalih reči u okolini.

Lematizacija predstavlja složeniji proces obrade teksta u odnosu na stemizaciju jer u procesu obrade podataka uzima širi kontekst i veći broj reči kako bi kreirao „lemu“. Lema predstavlja osnovnu formu reči, tj. oblik reči koji predstavlja njeno leksično značenje ili osnovni oblik. Lematizacija je proces redukcije reči pri čemu se sve varijacije određene reči svode na identičan leksički oblik.

Uklanjanje stop-reči

Stop-reči predstavljaju specifičan korpus reči svakog jezika koje sa sobom ne nose visoki značaj u obradi i analizi teksta. Ovakve reči je prilikom obrade teksta moguće izbaciti iz skupa ulaznih rečenica jer ne doprinose kvalitativnim svojstvima posmatranih tekstova, dok povećavaju vreme procesa obrade teksta. Ove reči se smatraju često korišćenim u jeziku i ne doprinose kontekstualnoj reprezentaciji ulaznih rečenica.

Stop-reči su uglavnom često korišćene reči, veznici, predlozi, rečce i sl. Primeri ovakvih reči u engleskom rečniku su sledeće: “the”, “a”, “and”, “is”, “in”, “into”...

U ovom odeljku su navedene samo neke od tehnika za preprocesiranje ulaznog teksta kako bi se prilagodio za dalju analizu i pripremu podataka za treniranje.

2.3.2. Priprema podataka za treniranje

Proces pripreme podataka za treniranje predstavlja postupak kreiranja statističkih parametara o skupu rečenica koje se obrađuju. Ulazni podaci ovog procesa predstavljaju preprocesirani podaci iz prethodnog poglavlja. Na ulazu se dobijaju obrađeni podaci u tekstualnom formatu koji se dalje prosleđuju u procesu vektorizacije pripreme podataka.

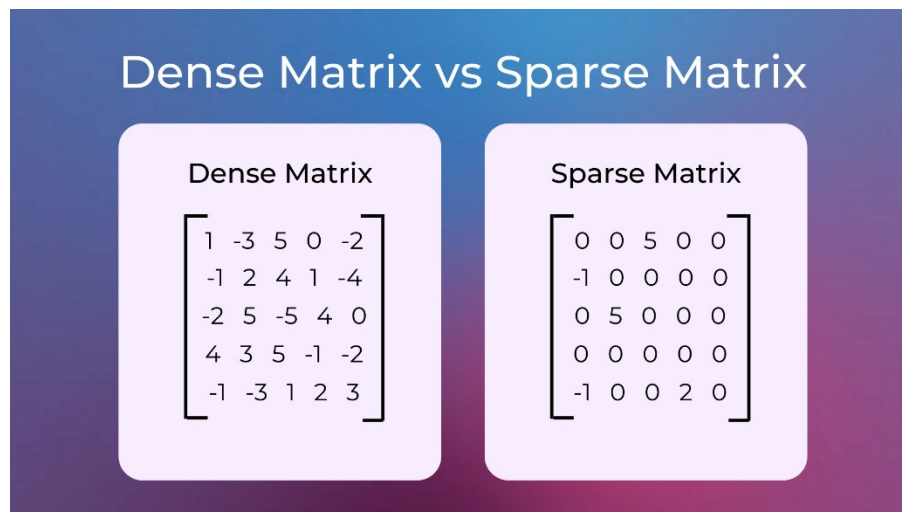
Vektorizacija teksta

Ovaj proces podrazumeva konverziju ulaznih tekstualnih podataka u numeričke podatke koje može računar da razume kako bi se dalje vršilo prilagođenje (eng. fitting). Numerički podaci se predstavljaju u obliku vektorizovanih elemenata matrice pri čemu svaki tekst predstavlja jedan red matrice. Ovakve matrice se nazivaju retko posednutim (eng. sparse matrix).



Na slici iznad prikazano je kako se ulazni tekstovi predstavljaju u obliku matrice. Svaki red ove matrice predstavlja pojedinačne ulazne rečenice, dok vrednosti matrice po kolonama predstavljaju broj ponavljanja datih reči u svakoj rečenici ponaosob. U slučaju kada je korpus rečenica izrazito veliki i različitog konteksta, vokabular korpusa će biti veći što će uticati na povećani broj kolona ove matrice. Ukoliko bi tekstovi bili različitih konteksta, dobili bismo veoma retko posednutu matricu kod koje bi velika većina vrednosti matrice imao vrednost 0.

Na slici ispod se može videti kako izgleda reprezentacija retko posednute matrice (eng. sparse matrix) u odnosu na gusto posednutu matricu (eng. dense matrix).



Prikaz reprezentacije gusto (levo) i retko (desno) posednutih matrica

Dva osnovna načina za vektorizaciju tekstova su:

- Bag of words
- Term frequency-Inverse document frequency

Bag of words (vreća reči) predstavlja pristup formiranja retko posednute matrice opisan u primeru prethodnog poglavlja gde se u matrici za svaku reč označava broj ponavljanja u svakoj rečenici pojedinačno. Ovaj način obrade ulaznih tekstova poseduje vokabular reči koji se dobija kao skup svih mogućih reči iz celokupnog ulaznog skupa podataka. Na osnovu vokabulara reči se zatim kreira mera ponavljanja određenih reči iz vokabulara.

Uz pomoć ovog pristupa se ne mogu zapamtiti kompleksnije zavisnosti između reči u rečenici poput strukture i redosleda, već se pamti samo informacija da li određena reč postoji ili ne u tekstovima.

Glavni problem ovog pristupa je dominantna učestalost određenih reči unutar dokumenta tj. ulaznog teksta pri čemu takve reči ne moraju nužno da nose dovoljan informativni kontekst. Kontekst dokumenta se zasniva na rečima koje su specifične domenu koji se obrađuje, a takve reči mogu imati manju učestalost u dokumentima.

Ovaj pristup predstavlja unapređenu verziju BOW pristupa pri čemu se u prvom delu obrade koristi rezultat dobijen BOW metodom. Termin Eng. Term frequency se odnosi na učestalost ponavljanja tokena unutar dokumenata i računa se kao broj ponavljanja određene reči podeljeno sa brojem reči u posmatranoj rečenici. Ovaj parametar predstavlja značajnost date reči u datom dokumentu.

Termin Eng. Inverse document frequency se odnosi na učestalost pojavljivanja datih reči u celokupnom skupu podataka. Konačni rezultat jeste proizvod dve dobijene vrednosti.

U sledećim formulama je prikazan detaljan način izračunavanja ovih vrednosti.

$$TF = \frac{\text{Number of times a word "X" appears in a Document}}{\text{Number of words present in a Document}}$$

$$IDF = \log \left(\frac{\text{Number of Documents present in a Corpus}}{\text{Number of Documents where word "X" has appeared}} \right)$$

$$TF-IDF(x, Document, Corpus) = TF(x, Document) \times IDF(x, Corpus)$$

Konačni rezultat ovog izračunavanja je retko posednuta matrica sa identičnim kolonama i vrstama kao u prethodnom slučaju, po kolonama su dokumenti/rečenice dok su po kolonama pojedinačne reči.

Vrednost za svaku reč predstavlja značajnost reči u datom dokumentu i u odnosu na celokupan korpus. Ona omogućava da na taj način dođu do izražaja reči specifične za kontekst posmatranog dokumenta.

Grupisanje reči pomoću n-gramova

Dodatni napredak u domenu kontekstualizacije dokumenata i hardverskih zahteva prilikom pripreme podataka za treniranje modela predstavlja grupisanje reči u N-gramove. N-gramovi predstavljaju sintagme reči u rečenici koje se odnose na sličan pojam. U dosadašnjem tekstu je zbog kompleksnosti u obradi tematike opisan rad nad 1-gram podacima, tj. kod vektorizacije tekstova se analizirala svaka reč kao zasebna jedinica (eng. gram).

Bitno je napomenuti da se mogu kreirati slučajevi sa 2-gram i 3-gram reprezentacijama pri čemu se skup 2 ili 3 reči posmatra kao zasebna jedinica u vektorizaciji tekstova. Ovakan korak u pripremi podataka može dodatno doprineti kontekstualizaciji vektora, smanjenju veličine retko posednute matrice i smanjenju vremena izvršenja samog procesa pripreme podataka i treniranja.

Problem sa 2 ili 3 reči u zasebnoj jedinici obrade jeste taj što se on primenjuje u slučajevima kada je korpus rečenica izrazito veliki jer je tada učestalost takvih sintagmi povećana u odnosu na mali korpus rečenica.



Primer 2, 3 i 4-gram izdvojenih jedinica za obradu

Navedene tehnike za obradu prirodnog jezika predstavljaju osnovne korake u pripremi skupa podataka koji se koristi za dalji proces treniranja jezičkih modela. U slučaju analize sentimenta, neophodni format je upravo vektorska matrica koja se dobija primenom vektorizacije nad preprocesiranim skupom podataka.

Skup podataka pripremljen na taj način je spreman za proces treniranja. U daljem tekstu izveštaja u delu praktičnog dela je detaljnije opisan postupak treniranja modela.

3. Praktični deo rada

Praktični deo ovog rada se sastoji od praktične implementacije koncepata koje se bave pribaljanjem, obradom i analizom podataka pri čemu se u završnom delu rada vrši klasifikacija skupa podataka sa ciljem dobijanja konačnih rezultata o analizi sentimenta odabranog skupa podataka. Praktična implementacija se sastoji od sledećih poglavlja:

- Učitavanje biblioteka
- Učitavanje i pregled osnovnog skupa podataka
- Preprocesiranje skupa podataka
- Eksplorativna analiza podataka (eng. EDA - Explanatory data analysis)
- Treniranje i evaluacija različitih modela u klasifikaciji

3.1. Učitavanje biblioteka

Za rad na praktičnom delu projekta neophodno je kreirati virtuelno okruženje u Python programskom jeziku i instalirati sve neophodne biblioteke za rad. Biblioteke koje se koriste su:

- ipykernel
- pandas
- matplotlib
- seaborn
- scikit-learn
- nltk
- transformers
- tensorflow
- torch

Navedene biblioteke je neophodno instalirati komandom `pip install <naziv biblioteke>`, a zatim sve neophodne module ovih biblioteka uključiti u projekat na početku jupyter radne sveske.

3.2. Skup podataka

Odabrani skup podataka nad kojim će se primeniti svi do sada obrađeni koncepti analize sentimenta jeste **Amazon Reviews: Unlocked Mobile Phones** koji sadrži više od 400 000 uzoraka podataka. Ovaj skup podataka opisuje recenzije kupljenih otključanih telefona preko sajta Amazon.com.

Dodatne informacije o skupu podataka se mogu pronaći na sledećem linku:

<https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones>

```
Učitavanje dataset-a

dataset = pd.read_csv("Amazon_Reviews.csv", index_col=False)
[3] ✓ 2.5s

Analiza dataset-a

dataset.head()
[4] ✓ 0.0s
...

```

	Product Name	Brand Name	Price	Rating	Reviews	Review Votes
0	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	I feel so LUCKY to have found this used (phone...	1.0
1	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	nice phone, nice up grade from my pantach revu...	0.0
2	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	Very pleased	0.0
3	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	It works good but it goes slow sometimes but i...	0.0
4	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	Great phone to replace my lost phone. The only...	0.0

```
dataset.describe(include='object')
[5] ✓ 0.4s
...

```

	Product Name	Brand Name	Reviews
count	413840	348669	413770
unique	4410	384	162490
top	Apple iPhone 4s 8GB Unlocked Smartphone w/ 8MP...	Samsung	Good
freq	1451	65747	2879

```
dataset.isna().sum()
[6] ✓ 0.0s
...

```

Product Name	0
Brand Name	65171
Price	5933
Rating	0
Reviews	70
Review Votes	12296
dtype:	int64

Učitavanje i pregled skupa podataka

Skup podataka se sastoji atributa Product Name, Brand Name, Price, Rating, Reviews, Review Votes. Za proces analize sentimenata će biti od koristi atributi Reviews i Rating. Iz osnovnog pregleda skupa podataka, moguće je utvrditi da veliki broj podataka nema popunjene podatke za određene attribute i da treba obratiti pažnju prilikom preprocesiranja i filtriranja skupa podataka kako bi se sačuvao najveći broj uzoraka skupa podataka koji imaju validne vrednosti za dva atributa od značaja.

3.3. Preprocesiranje podataka

U praktičnom delu rada prilikom preprocesiranja podataka primenjene su sledeće tehnike:

- Izdvajanje značajnih atributa
- Enkodiranje Rating atributa
- Izbacivanje redova bez popunjenih vrednosti (oko 70 redova)
- Izbacivanje URL tagova iz ulaznih tekstova
- Izbacivanje karaktera koji nisu alfa-numerički
- Prebacivanje tekstova u mala slova (eng. lower case)

- Tokenizacija tekstova
- Izbacivanje stop reči
- Lematizacija (ili stemovanje)
- Izbacivanje redova sa više od 2500 tokena u tekstualnoj recenziji
- Skraćivanje skupa podataka na manji broj pogodan za treniranje modela

Na sledećim slikama je prikazana praktična implementacija pojedinih tehnika za preprocesiranje tekstova:

Enkodiranje rating feature-a

```
def encode_rating(rating: int) -> str:
    if rating >= 4:
        return "Positive"
    elif rating <= 2:
        return "Negative"
    else:
        return "Neutral"
```

[8] ✓ 0.0s

Enkodiranje rating feature-a

✓ Čišćenje review-a

```
def removing_url(review: str) -> str:
    return re.sub(r'http\S+', '', review)

def clean_non_alphanumeric(review: str) -> str:
    return re.sub('[^a-zA-Z]', ' ', review)

def convert_to_lowercase(review: str) -> str:
    return str(review).lower()

def tokenize_text(review_text: str) -> str:
    return word_tokenize(review_text)
```

[13] ✓ 0.0s

Čišćenje teksta

Uklanjanje engleskih stop reči i lematizacija

```
def remove_stopwords(token: str) -> list[str]:
    stop_words = set(stopwords.words('english'))
    return [item for item in token if item not in stop_words]

def lemmatization(review_text: str) -> list[str]:
    lemma = WordNetLemmatizer()
    return [lemma.lemmatize(word=token, pos='v') for token in review_text]
```

[16] ✓ 0.0s

Uklanjanje stop reči i lematizacija

3.4. Eksplorativna analiza podataka

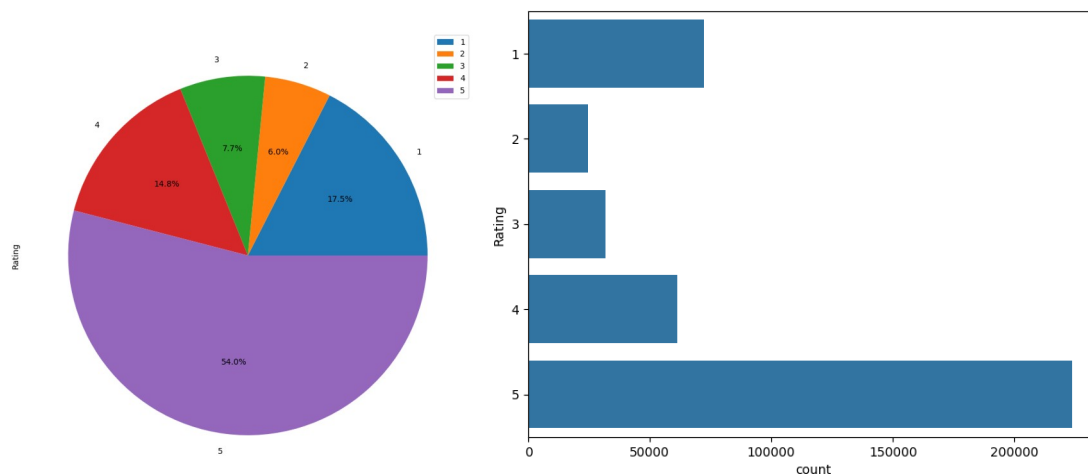
Eksplorativna analiza podataka služi kako bi se putem različitim grafičkih alata omogućila bolja vizualizacija podataka, raspodela vrednosti po posmatranim atributima, učestalost određenih tokena u skupu podataka, distribucija dužina rečenica u skupu podataka i sl.

3.4.1. Vizualizacija raspodele sentimenata

Vizualizacija raspodele sentimenata, tj. kategoričkog atributa koji opisuje kojoj kategoriji sentimenta posmatrani tekst pripada, prikazuje učešće određenih kategoričkih atributa u celokupnom skupu podataka i broj ponavljanja.

Vizualizacija ovakvih podataka je obavljena uz pomoć `pd.DataFrame.plot.pie` i `sns.countplot` pri čemu je u zavisnosti od pristupa potrebno prilagoditi ulazne podatke na određeni način.

Na vizualizaciji je prikazana raspodela ne enkodiranih vrednosti Rating atributa, kao i enkodiranog Label atributa.

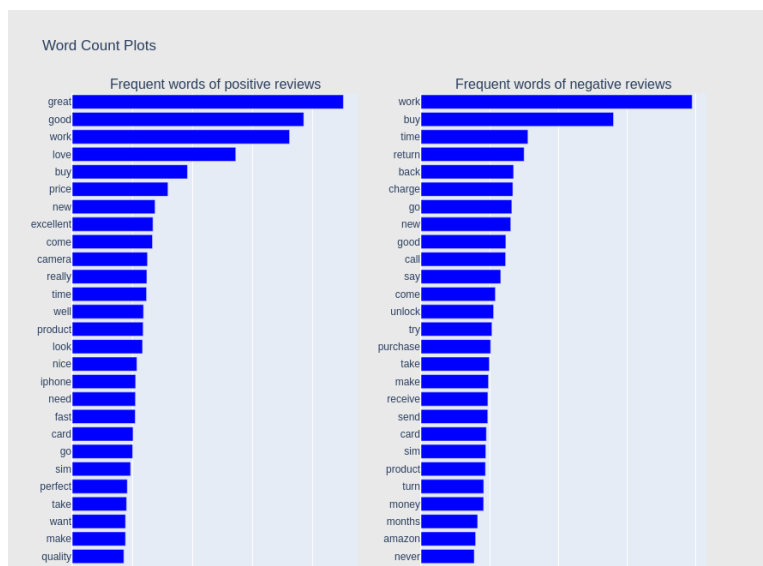


Grafički prikaz udela pojedinačnih vrednosti Rating atributa pre faze enkodiranja

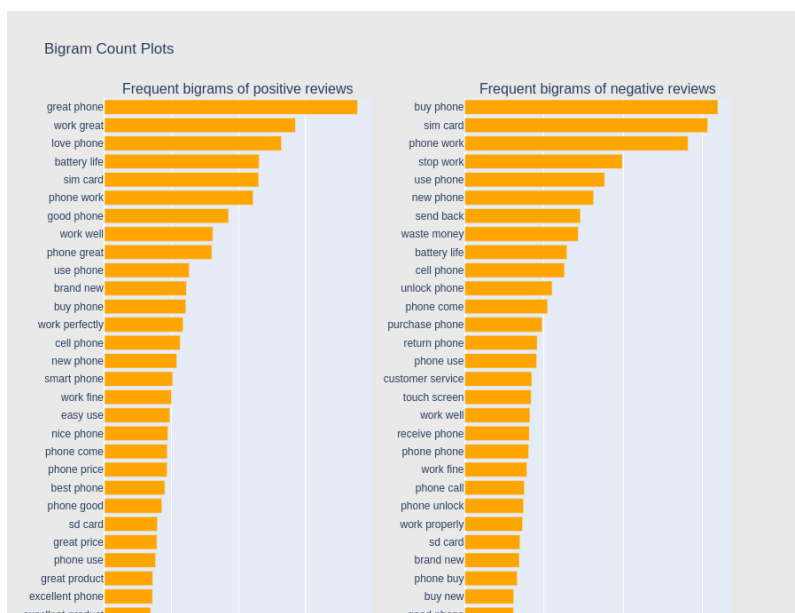
object dtype
charged second 1 work port Length used disappointed looks Name Item
normal wear Reviews
know loose hard Tha
good already really stay
said junk listed
new Convoy loved ser owned
solder got
problems charging

3.4.3. N-gram vizuelizacija

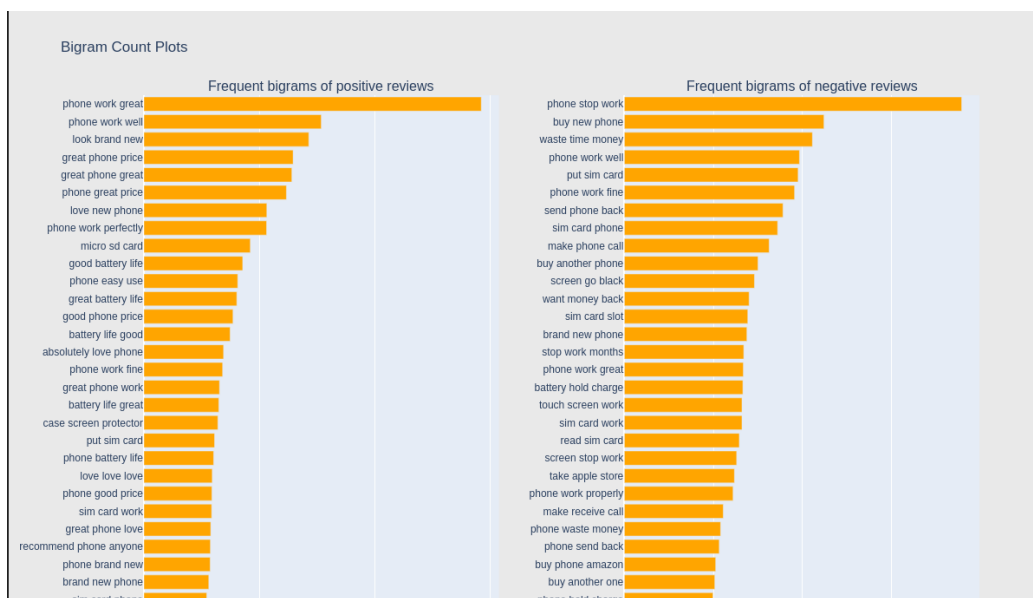
Na sledećim graficima su prikazani rezultati ovakve vizualizacije:



1-gram uзорci sa najvećim brojem pojavljivanja



2-gram uzorci sa najvećim brojem pojavljivanja



3-gram uzorci sa najvećim brojem pojavljivanja

Na datim slikama je moguće primetiti kako se pozitivni ili negativni kontekst sa različitim frekvencijama nalazi u datim podskupima podataka. Može se uočiti pozitivan kontekst 1-gram, 2-gram i 3-gram uzoraka u pozitivnom podskupu (levo), kao i identična zavisnost kod negativnih slučajeva u negativnom podskupu (desno). Iz navedenih rezultata može se dalje analizirati uticaj na pozitivno ili negativno zadovoljstvo korisnika koji su ostavljali svoja zapažanja i kritike.

3.5. Priprema podataka za treniranje modela

U fazi pripreme podataka u praktičnom delu rada izvršena je vektorizacija podataka kao poslednji korak u pripremi podataka za treniranje modela i klasifikacione algoritme. U samom procesu vektorizacije tekstova korišćene su obe metode vektorizacije podataka prethodno opisane u teorijskom delu ovom izveštaja.

U praktičnom delu vektorizacija podataka je urađena za dokumente korpusa pri čemu je korišćena i 1-gram i 3-gram reprezentacija tekstova.

Korišćeni objekti za vektorizaciju tekstova su:

- CountVectorizer
- TfidfTransformer
- TfidfVectorizer

3.5.1. Vektorizacija tekstova korišćenjem 1-gram reprezentacija

Na narednim slikama je prikazan način za pripremu podataka nad 1-gram reprezentacijama.

```
TfidfTransformer

tfidf_transform = TfidfTransformer(norm=None)
X_tr_tfidf = tfidf_transform.fit_transform(X_tr_bow)
X_te_tfidf = tfidf_transform.transform(X_te_bow)

y_tfidf_transform = TfidfTransformer(norm=None)

y_tr_transformed_using_TfidfTransformer = y_tfidf_transform.fit_transform(y_tr_transformed_using_bow)
y_te_transformed_using_TfidfTransformer = y_tfidf_transform.transform(y_te_transformed_using_bow)

print("Shape of X_tr_tfidf: ", X_tr_tfidf.shape)
print("Shape of X_te_tfidf: ", X_te_tfidf.shape)

print("Shape of y_tr_transformed_using_TfidfTransformer: ", y_tr_transformed_using_TfidfTransformer.shape)
print("Shape of y_te_transformed_using_TfidfTransformer: ", y_te_transformed_using_TfidfTransformer.shape)

Shape of X_tr_tfidf: (7000, 6442)
Shape of X_te_tfidf: (3000, 6442)
Shape of y_tr_transformed_using_TfidfTransformer: (7000, 3)
Shape of y_te_transformed_using_TfidfTransformer: (3000, 3)
```

Vektorizacija 1-gram reprezentacija tekstova korišćenjem TfidfTransformer

```
CountVectorizer

> ~
bow_transform = CountVectorizer(lowercase=False)
X_tr_bow = bow_transform.fit_transform(training_data['CleanReviews'])
y_tr = training_data['Label']

y_bow_transform = CountVectorizer(lowercase=False)

y_tr_transformed_using_bow = y_bow_transform.fit_transform(training_data['Label'])

print("Shape of X_tr_bow: ", X_tr_bow.shape)
print("Shape of y_tr: ", y_tr.shape)

print("len(bow_transform.vocabulary_): ", len(bow_transform.vocabulary_))

X_te_bow = bow_transform.transform(test_data['CleanReviews'])
y_te = test_data['Label']

y_te_transformed_using_bow = y_bow_transform.transform(test_data['Label'])

print("Shape of X_te_bow: ", X_te_bow.shape)
print("Shape of y_te: ", y_te.shape)

print("Shape of y_tr_transformed_using_bow: ", y_tr_transformed_using_bow.shape)
print("Shape of y_te_transformed_using_bow: ", y_te_transformed_using_bow.shape)

[35]
... Shape of X_tr_bow: (7000, 6442)
Shape of y_tr: (7000,)
len(bow_transform.vocabulary_): 6442
Shape of X_te_bow: (3000, 6442)
Shape of y_te: (3000,)
Shape of y_tr_transformed_using_bow: (7000, 3)
Shape of y_te_transformed_using_bow: (3000, 3)
```

Vektorizacija 1-gram reprezentacija tekstova korišćenjem CountVectorizer

```
TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()
X_tfidf_tr = tfidf_vectorizer.fit_transform(training_data['CleanReviews'])
X_tfidf_te = tfidf_vectorizer.transform(test_data['CleanReviews'])

y_tfidf_vectorizer = TfidfVectorizer()
y_tr_transformed_using_TfidfVectorizer = y_tfidf_vectorizer.fit_transform(training_data['Label'])
y_te_transformed_using_TfidfVectorizer = y_tfidf_vectorizer.transform(test_data['Label'])

print("Shape of X_tfidf_tr: ", X_tfidf_tr.shape)
print("Shape of X_tfidf_te: ", X_tfidf_te.shape)

print("Shape of y_tr_transformed_using_TfidfVectorizer: ", y_tr_transformed_using_TfidfVectorizer.shape)
print("Shape of y_te_transformed_using_TfidfVectorizer: ", y_te_transformed_using_TfidfVectorizer.shape)

1
Shape of X_tfidf_tr: (7000, 6442)
Shape of X_tfidf_te: (3000, 6442)
Shape of y_tr_transformed_using_TfidfVectorizer: (7000, 3)
Shape of y_te_transformed_using_TfidfVectorizer: (3000, 3)
```

Vektorizacija 1-gram reprezentacija tekstova korišćenjem TfidfVectorizer

Kao što je i u teorijskom delu izveštaja opisano, primenom vektorizacije podataka se dobija retko posednuta matrica. Primeri takvih matrica nad ulaznim skupom od 5 tekstova su prikazani na sledećoj slici.

Reprezentacija 1-grama korišćenjem TF-IDF transformacije nad rezultatima bag of Words transformacije

```
tfidf_transform = TfidfTransformer(norm='lone')
X = tfidf_transform.fit_transform(X)
pd.DataFrame(X.toarray(), columns=bow_vectorizer.get_feature_names_out())
```

	accurate	align	allow	around	arrive	average	base	battery	bite	bounce	...	tray	try	type	typically	units	use	user	well	work	yes
0	0.000000	0.000000	0.000000	0.000000	1.693147	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.405465	0.000000
1	0.000000	2.098612	2.098612	2.098612	0.000000	2.098612	0.000000	1.693147	0.000000	2.098612	...	2.098612	2.098612	2.098612	2.098612	2.098612	2.098612	0.000000	0.000000	0.000000	
2	0.000000	0.000000	0.000000	0.000000	1.693147	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.405465	
3	2.098612	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
4	0.000000	0.000000	0.000000	0.000000	0.000000	2.098612	5.079442	2.098612	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	2.098612	0.000000	0.000000	4.197225	2.810930	

5 rows x 23 columns

Reprezentacija 1-grama pomoću TfidfVectorizer-a bez korišćenja IDF dela

```
vectorizer = TfidfVectorizer(use_idf=False)
X = vectorizer.fit_transform(training_data['CleanReviews'][:5])
pd.DataFrame(X.toarray(), columns=bow_vectorizer.get_feature_names_out())
```

	accurate	align	allow	around	arrive	average	base	battery	bite	bounce	...	tray	try	type	typically	units	use	user	well	work	yes
0	0.000000	0.000000	0.000000	0.000000	0.250000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.250000	0.000000	
1	0.000000	0.108465	0.108465	0.000000	0.108465	0.000000	0.108465	0.000000	0.108465	0.000000	...	0.108465	0.108465	0.108465	0.108465	0.108465	0.108465	0.000000	0.000000	0.000000	
2	0.000000	0.000000	0.000000	0.000000	0.577350	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.577350	0.000000	
3	0.447214	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
4	0.000000	0.000000	0.000000	0.000000	0.000000	0.141421	0.424264	0.141421	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.141421	0.000000	0.000000	0.282843	0.282843	

5 rows x 23 columns

Reprezentacija 1-grama pomoću TfidfVectorizer-a sa uključnim IDF delom

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(training_data['CleanReviews'][:5])
pd.DataFrame(X.toarray(), columns=bow_vectorizer.get_feature_names_out())
```

	accurate	align	allow	around	arrive	average	base	battery	bite	bounce	...	tray	try	type	typically	units	use	user	well	work	yes
0	0.000000	0.000000	0.000000	0.000000	0.214109	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.177730	0.000000	
1	0.000000	0.109271	0.109271	0.109271	0.109271	0.000000	0.088160	0													

Primeri retko posednutih matrica kreiranih na tri različita načina

3.5.2. Vektorizacija tekstova korišćenjem 3-gram reprezentacija

Na narednim slikama je prikazan način za pripremu podataka nad 3-gram reprezentacijama.

```

Prpimea podataka

CountVectorizer

bow_transform = CountVectorizer(tokenizer=lambda doc: doc.split(), ngram_range=(3,3), lowercase=False)
X_tr_bow = bow_transform.fit_transform(training_data['CleanReviews'])
y_tr = training_data['Label']

print("Shape of X_tr_bow: ", X_tr_bow.shape)
print("Shape of y_tr: ", y_tr.shape)

print("len(bow_transform.vocabulary): ", len(bow_transform.vocabulary))

X_te_bow = bow_transform.transform(test_data['CleanReviews'])
y_te = test_data['Label']

print("Shape of X_te_bow: ", X_te_bow.shape)
print("Shape of y_te: ", y_te.shape)

[49]

... c:\Users\katar\OneDrive\Desktop\Master\Duboko učenje\Projekat\venv\Lib\site-packages\sklearn\feature_extraction\tfidf.py:517: UserWarning:
The parameter 'token_pattern' will not be used since 'tokenizer' is not None

Shape of X_tr_bow: (7000, 90493)
Shape of y_tr: (7000,)
len(bow_transform.vocabulary): 90493
Shape of X_te_bow: (3000, 90493)
Shape of y_te: (3000,)

```

Vektorizacija 3-gram reprezentacija tekstova korišćenjem CountVectorizer

TfidfTransformer

```
tfidf_transform = TfidfTransformer(norm=None)
X_tr_tfidf = tfidf_transform.fit_transform(X_tr_bow)
X_te_tfidf = tfidf_transform.transform(X_te_bow)

print("Shape of X_tr_tfidf: ", X_tr_tfidf.shape)
print("Shape of X_te_tfidf: ", X_te_tfidf.shape)
```

[50]

```
... Shape of X_tr_tfidf: (7000, 90493)
Shape of X_te_tfidf: (3000, 90493)
```

Vektorizacija 3-gram reprezentacija tekstova korišćenjem TfidfTransformer-a

TfidfVectorizer

```
tfidf_vectorizer = TfidfVectorizer(use_idf=True, tokenizer=lambda doc: doc.split(), ngram_range=(3,3))
X_tfidf_tr = tfidf_vectorizer.fit_transform(training_data['CleanReviews'])
X_tfidf_te = tfidf_vectorizer.transform(test_data['CleanReviews'])

print("Shape of X_tfidf_tr: ", X_tfidf_tr.shape)
print("Shape of X_tfidf_te: ", X_tfidf_te.shape)
```

[51]

```
... Shape of X_tfidf_tr: (7000, 90493)
Shape of X_tfidf_te: (3000, 90493)
```

Vektorizacija 3-gram reprezentacija tekstova korišćenjem TfidfVectorizer-a

Reprezentacija 3-grama korišćenjem TF-IDF transformacije nad rezultatima Bag of Words transformacije

```
tfidf_transform = TfidfTransformer(norm=None)
X = tfidf_transform.fit_transform(X)
df = pd.DataFrame(X.toarray(), columns=bow_vectorizer.get_feature_names_out())
```

	allow secure fit	around post pocket	arrive work expect	average guy large	base shaky battery	battery hold charge	battery life good	battery life power	battery regularly doesn't	bounce around post	...	use w one	user call data	w one hand	w thumb small	well except battery	well lose power	work great arrive	work much longer	work well except	yes calibrate battery
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2.098612	0.000000	0.000000	0.000000
1	2.098612	2.098612	0.000000	0.000000	0.000000	0.000000	2.098612	0.000000	0.000000	2.098612	...	2.098612	2.098612	2.098612	2.098612	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
2	0.000000	0.000000	2.098612	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
3	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
4	0.000000	0.000000	0.000000	0.000000	2.098612	2.098612	0.000000	2.098612	2.098612	0.000000	...	0.000000	0.000000	0.000000	0.000000	2.098612	2.098612	0.000000	2.098612	2.098612	2.098612

5 rows x 117 columns

Reprezentacija 3-grama pomoću TfidfVectorizer-a bez korišćenja IDF dela

```
vectorizer = TfidfVectorizer(use_idf=False, tokenizer=lambda doc: doc.split(), ngram_range=(3,3))
X = vectorizer.fit_transform(training_data['CleanReviews'])
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df
```

Warning: UserWarning: The parameter "token_pattern" will not be used since "tokenizer" is not None

	allow secure fit	around post pocket	arrive work expect	average guy large	base shaky battery	battery hold charge	battery life good	battery life power	battery regularly doesn't	bounce around post	...	use w one	user call data	w one hand	w thumb small	well except battery	well lose power	work great arrive	work much longer	work well except	yes calibrate battery
0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.267	0.000	0.000	0.000
1	0.124	0.124	0.0	0.124	0.000	0.000	0.124	0.000	0.000	0.124	...	0.124	0.124	0.124	0.124	0.000	0.000	0.000	0.000	0.000	0.000
2	0.000	0.000	1.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
3	0.000	0.000	0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
4	0.000	0.000	0.0	0.000	0.171	0.171	0.000	0.171	0.171	0.000	...	0.000	0.000	0.000	0.000	0.171	0.171	0.000	0.171	0.171	0.171

5 rows x 117 columns

Reprezentacija 3-grama pomoću TfidfVectorizer-a sa uključenim IDF delom

```
vectorizer = TfidfVectorizer(use_idf=True, tokenizer=lambda doc: doc.split(), ngram_range=(3,3))
X = vectorizer.fit_transform(training_data['CleanReviews'])
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df
```

Warning: UserWarning: The parameter "token_pattern" will not be used since "tokenizer" is not None

	allow secure fit	around post pocket	arrive work expect	average guy large	base shaky battery	battery hold charge	battery life good	battery life power	battery regularly doesn't	bounce around post	...	use w one	user call data	w one hand	w thumb small	well except battery	well lose power	work great arrive	work much longer	work well except	yes calibrate battery
0	0.000	0.000	0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.267	0.000	0.000	0.000
1	0.124	0.124	0.0	0.124	0.000	0.000	0.124	0.000	0.000	0.124	...	0.124	0.124	0.124	0.124	0.000	0.000	0.000	0.000	0.000	0.000
2	0.000	0.000	1.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
3	0.000	0.000	0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	...	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
4	0.000	0.000	0.0	0.000	0.171	0.171	0.000	0.171	0.171	0.000	...	0.000	0.000	0.000	0.000	0.171	0.171	0.000	0.171	0.171	0.171

5 rows x 117 columns

Primeri retko posednutih matrica kreiranih na tri različita načina

3.6. Treniranje i evaluacija modela

Proces treniranja i evaluacije modela predstavlja proces pri kojem se vrši obučavanje metoda mašinskog učenja, modela baziranog na transformerskoj arhitekturi ili veštačkih neuronskih mreža kako bi se postigla visoka preciznost u procesu evaluacije. Pre početka samog toka procesa treniranja, pripremljeni podaci se dele na ulazne i izlazne podatke. Svi ulazni i izlazni podaci se dele na tri grupe podataka: trening, evaluacione i test podatke.

U našem slučaju analize sentimenata ulazni podaci predstavljaju pripremljene tekstualne recenzije u formatu retko posednutih matrica ukoliko treniramo metodu mašinskom učenja, ili sami preprocesirani tekstovi u slučaju da vršimo evaluaciju već treniranih modela.

Evaluacija je proces određivanja uspešnosti algoritma da obavi analizu sentimenata, tačnije da klasifikuje ulazni tekst u jednu od ponuđenih kategorija. Uspešnosti modela se ocenjuje preciznošću (eng. accuracy) koja nosi vrednost u opsegu od 0 do 100% tačnosti.

3.6.1. Treniranje i evaluacija modela mašinskog učenja

U toku treniranja metoda mašinskog učenja primenjuje se 1-gram i 3-gram metode pripreme podataka i u dva nezavisna toka Jupyter radne sveske se vrši prilagođavanje (eng. fitting) / treniranje i evaluacija dobijenih rešenja.

U procesu su uključene sledeće metode:

- Logistic Regression
- Multinomial Naive Bayes
- Random Forest Classifier
- SVC

Za svaku od metoda se primenjuje prilagođavanje na tri ulazne retko posednute matrice dobijene metodama:

- Bag of words
- Tf-idf Transformer
- Tf-idf Vectorizer

Za svaki od rezultata se prikazuje matrica konfuzije i vrednost tačnosti algoritma. Svi rezultati tačnosti algoritama se agregiraju i prikazuju u konačnom pregledu rezultata.

Na sledećim slikama je prikazan primer izvornog koda prilagođavanja nekih od metoda, kao i rezultati tačnosti metoda.

```

Logistic Regression

from sklearn.linear_model import LogisticRegression

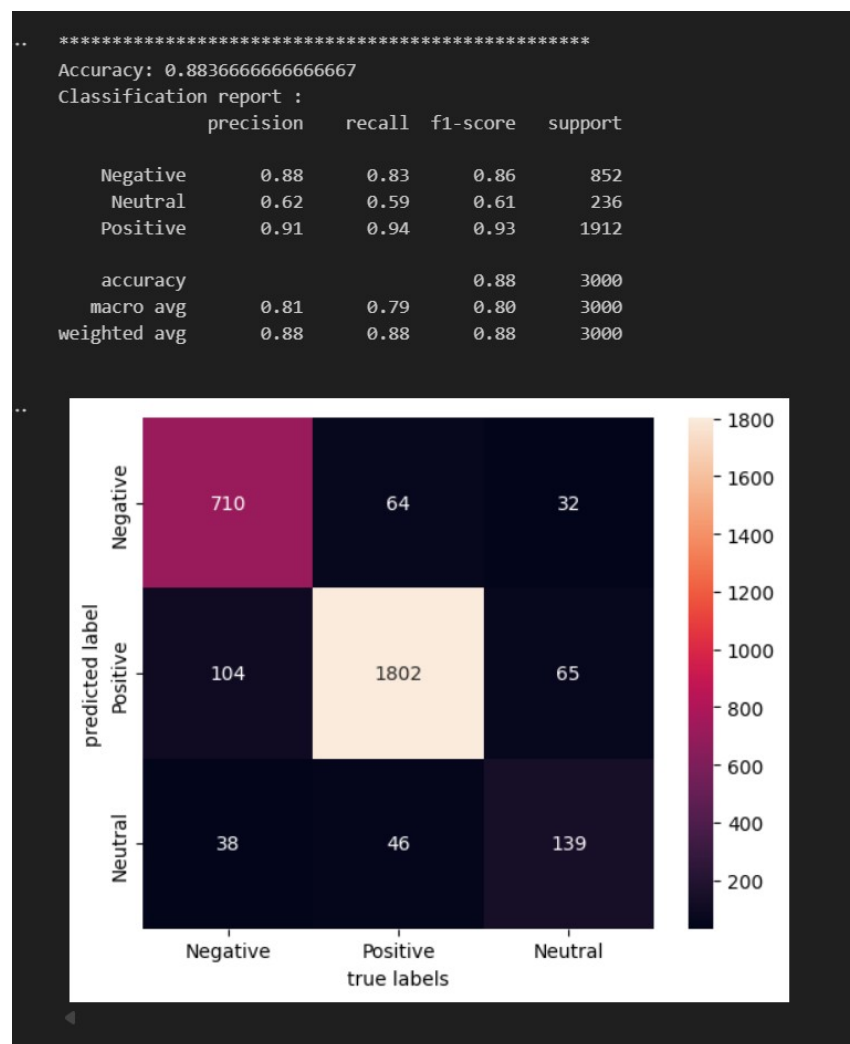
model_name = "LogisticRegression_1_gram"
model = LogisticRegression(max_iter=500)

dataframe_bow = simple_logistic_classify(model, f'{model_name}_{BOW_METHOD_STRING}', X_tr_bow, y_tr, X_te_bow, y_te, BOW_METHOD_STRING)
dataframe_tfidf_transformer = simple_logistic_classify(model, f'{model_name}_{TF_IDF_TRANSFORMER_METHOD_STRING}', X_tr_tfidf, y_tr, X_te_tfidf, y_te, TF_IDF_TRANSFORMER_METHOD_STRING)
dataframe_tfidf_vectorizer = simple_logistic_classify(model, f'{model_name}_{TF_IDF_VECTORIZER_METHOD_STRING}', X_tr_tfidf, y_tr, X_te_tfidf, y_te, TF_IDF_VECTORIZER_METHOD_STRING)

data_table_logistic_regression_1_gram = pd.concat([dataframe_bow, dataframe_tfidf_transformer, dataframe_tfidf_vectorizer], ignore_index=True)
data_table_logistic_regression_1_gram

```

Prilagođavanje logističke regresije na podatke pripremljene 1-gram metodom



Evaluacija rezultata logističke regresije

Ukupno je urađeno 30 prilagođavanja sa 4 metoda mašinskog učenja, sa 2 metode pripremljenih ulaznih podataka (1-gram i 3-gram) i sa 3 metode vektorizacije podataka (BOW, Tf-idf Transformer, Tf-idf Vectorizer).

Konačni rezultati tačnosti svih prilagođavanja su sledeći:

	model	accuracy
8	RandomForestClassifier_1_gram_tf-idf-vectorizer	0.898333
6	RandomForestClassifier_1_gram_bow	0.897000
7	RandomForestClassifier_1_gram_tf-idf-transformer	0.896667
1	LogisticRegression_1_gram_tf-idf-transformer	0.883667
0	LogisticRegression_1_gram_bow	0.878333
9	SVC_1_gram_bow	0.875000
11	SVC_1_gram_tf-idf-vectorizer	0.874000
10	SVC_1_gram_tf-idf-transformer	0.867333
2	LogisticRegression_1_gram_tf-idf-vectorizer	0.854333
3	MultinomialNB_1_gram_bow	0.844667
4	MultinomialNB_1_gram_tf-idf-transformer	0.827000
15	MultinomialNB_3_gram_bow	0.826667
16	MultinomialNB_3_gram_tf-idf-transformer	0.809000
20	RandomForestClassifier_3_gram_tf-idf-vectorizer	0.805667
18	RandomForestClassifier_3_gram_bow	0.805000
19	RandomForestClassifier_3_gram_tf-idf-transformer	0.805000
13	LogisticRegression_3_gram_tf-idf-transformer	0.802000
5	MultinomialNB_1_gram_tf-idf-vectorizer	0.799333
22	SVC_3_gram_tf-idf-transformer	0.793667
12	LogisticRegression_3_gram_bow	0.791667
21	SVC_3_gram_bow	0.791667
23	SVC_3_gram_tf-idf-vectorizer	0.786000
17	MultinomialNB_3_gram_tf-idf-vectorizer	0.761667
14	LogisticRegression_3_gram_tf-idf-vectorizer	0.758000

Evaluacija svih rezultata tačnosti

Iz dobijenih rezultata može se uočiti da je najbolje rezultate klasifikacije analize sentimenata postigao Random Forest Classifier prilagođen nad 1-gram pripremljenim podacima pri čemu su za sva tri tipa vektorizacije rezultati približno simetrični.

3.6.2. Treniranje i evaluacija modela zasnovanih na BERT arhitekturi

U procesu korišćenja modela baziranih na BERT arhitekturi, iskorišćeni su postojeći trenirani modeli nad drugim identičnim ili približno identičnim skupovima podataka za analizu sentimenata. U ovom delu praktičnog rada su korišćeni sledeći modeli preuzeti sa HuggingFace sajta:

- nlptown/bert-base-multilingual-uncased-sentiment
- cardiffnlp/twitter-roberta-base-sentiment-latest
- distilbert-base-uncased-finetuned-sst-2-english

Treba napomenuti da je kod evaluacije ovih modela neophodno primeniti mapiranje rezultata predikcije datog modela na rezultate ulaznog skupa podataka kako bi bilo moguće izračunati tačnost modela.

```
model_name = "nlptown/bert-base-multilingual-uncased-sentiment"
sentiment_pipeline = pipeline("text-classification", model=model_name)

dataset_copy = finetuning_reduced_filtered_dataset.copy()

data = dataset_copy['CleanReviews'].tolist()

result_map = {'5 stars': 'Positive', '4 stars': 'Positive', '3 stars': 'Neutral', '2 stars': 'Negative', '1 star': 'Negative'}

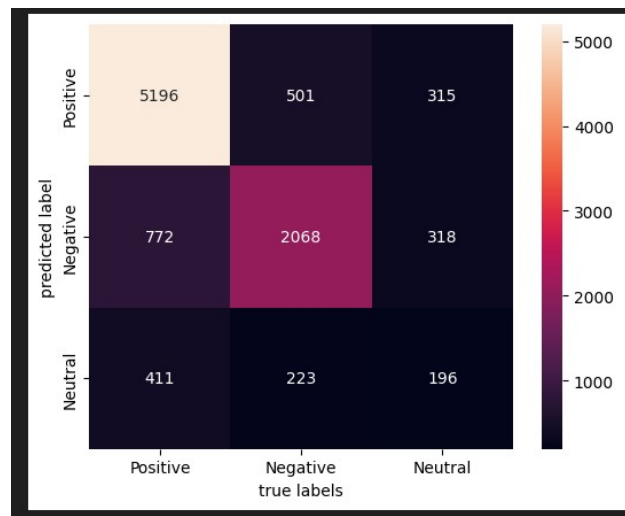
results = sentiment_pipeline(data)
dataset_copy['label_predictions'] = [result_map[result['label']] for result in results]

dataframe_bert = calculateResultsAndReturnDataframe(model_name, dataset_copy, dataset_copy['Label'].unique())
```

✓ 7m 25.4s

Device set to use cpu
Accuracy: 0.746
Classification report :

	precision	recall	f1-score	support
Negative	0.65	0.74	0.70	2792
Neutral	0.24	0.24	0.24	829
Positive	0.86	0.81	0.84	6379
accuracy			0.75	10000
macro avg	0.59	0.60	0.59	10000
weighted avg	0.75	0.75	0.75	10000



Primer mapiranja i evaluacije pretreniranih modela

Nakon evaluacije svih pretreniranih modela, dobija se sledeći rezultat:

```
final_dataframe_bert = pd.concat([dataframe_bert, dataframe_roberta,dataframe_distilbert ], ignore_index=True)
final_dataframe_bert_sorted = final_dataframe_bert.sort_values(by="accuracy", ascending=False)
final_dataframe_bert_sorted
```

✓ 0.0s

	model	accuracy
0	nlptown/bert-base-multilingual-uncased-sentiment	0.746000
2	distilbert-base-uncased-finetuned-sst-2-english	0.737324
1	cardiffnlp/twitter-roberta-base-sentiment-latest	0.678600

Tačnost pretreniranih modela

Iz ovih rezultata, moguće je pretpostaviti da se dosta razlikuju skupovi podataka nad kojima su obučavani navedeni modeli u odnosu na skup podataka koji se obrađuje u ovom radu te stoga i rezultati nisu zadovoljavajući u ovim primerima. Takođe, neki od ovih modela imaju drugačiji sistem ocenjivanja sentimenta (pr. od 1 do 5) te stoga i mapiranje može dodatno uticati na rezultate ove evaluacije.

3.6.3. Treniranje i evaluacija veštačkih neuronskih mreža

Priprema podataka

Sledeći deo koda prikazuje pripremu tekstualnih podataka za neuronsku mrežu:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.preprocessing import LabelEncoder

MAX_LEN = 100
VOCAB_SIZE = 20000
EMBEDDING_DIM = 100

tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>")
tokenizer.fit_on_texts(filtered_dataset['CleanReviews'])

X = tokenizer.texts_to_sequences(filtered_dataset['CleanReviews'])
X = pad_sequences(X, maxlen=MAX_LEN, padding='post', truncating='post')

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(filtered_dataset['Label'])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

Priprema podataka za veštačke neuronske mreže

Tekst iz kolone CleanReviews se pomoću Tokenizer-a pretvara u numeričke sekvence, a zatim se sekvence izjednačavaju na istu dužinu funkcijom pad_sequences. Oznake (Label) se

kodiraju u brojeve pomoću LabelEncoder, a ceo skup se deli na podatke za treniranje i testiranje pomoću train_test_split. Tako pripremljeni podaci mogu se direktno koristiti za obuku modela dubokog učenja.

Treniranje modela

U ovom delu rada korišćeno je više različitih tipova neuronskih mreža za analizu sentimenta teksta. Sve mreže su izrađene pomoću biblioteke TensorFlow Keras, koristeći sekvencijalni pristup i slojeve za obradu teksta, kao što su Embedding, LSTM, GRU, Conv1D i Bidirectional. Svaka mreža je trenirana sa istim skupom parametara i istim podacima, čime je omogućeno objektivno poređenje rezultata.

Prva mreža je LSTM (Long Short-Term Memory) model, koji sadrži sloj Embedding za pretvaranje reči u vektore, zatim LSTM sloj sa 128 neurona, dropout vrednosti 0.2 i dodatnim recurrent_dropout-om za sprečavanje pretreniravanja. Nakon LSTM sloja dodati su Dense slojevi sa ReLU i Softmax aktivacijama, gde prvi uči složenije reprezentacije, a drugi daje izlazne verovatnoće po klasama. Ovaj model je izabran jer može dugoročno pamtit i kontekst i odnose između reči, što je ključno za analizu sentimenta.

Druga mreža je BiLSTM (Bidirectional LSTM), koja koristi isti princip kao LSTM, ali obrađuje sekvencu u oba smera – od početka i od kraja rečenice. Na taj način model ima pristup i prethodnom i budućem kontekstu svake reči, što omogućava dublje razumevanje značenja teksta. Ova struktura je korisna kod složenih rečenica gde se značenje ne može razumeti samo iz redosleda reči unapred.

Treća mreža je GRU (Gated Recurrent Unit), koja je konstruisana slično kao LSTM, ali sa pojednostavljenom arhitekturom koja ima manje parametara. GRU koristi samo dve kapije – reset i update – čime brže uči i troši manje resursa, a i dalje zadržava sposobnost pamćenja dugoročnih zavisnosti. Zbog toga je pogodna za brže treniranje i eksperimentisanje sa većim skupovima podataka.

Četvrta mreža je CNN (Convolutional Neural Network) model, koji koristi konvolucion i sloj Conv1D sa 128 filtera i aktivacijom ReLU, praćen slojem GlobalMaxPooling1D koji smanjuje dimenzionalnost i izdvaja najvažnije osobine iz svake sekvence. Na kraju se koriste Dense slojevi sa ReLU i Softmax funkcijama za klasifikaciju. CNN mreža je izabrana jer je efikasna u prepoznavanju lokalnih obrazaca u tekstu, kao što su fraze ili kombinacije reči koje nose emocionalno značenje.

Sledeće slike prikazuju delove koda koji definišu neke od pomenutih modela, kao i neke rezultate dobijene njihovim treniranjem i testiranjem.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

model_name = "LSTM_sentiment_model"

model_lstm = Sequential([
    Embedding(VOCAB_SIZE, EMBEDDING_DIM, input_length=MAX_LEN),
    LSTM(128, dropout=0.2, recurrent_dropout=0.2),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(3, activation='softmax')
])

model_lstm.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
history_lstm = model_lstm.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=3, batch_size=128, verbose=1)

y_pred = np.argmax(model_lstm.predict(X_test), axis=1)

dataset_copy = pd.DataFrame({
    'Label': label_encoder.inverse_transform(y_test),
    'label_predictions': label_encoder.inverse_transform(y_pred)
})

dataframe_lstm = calculateResultsAndReturnDataframe(model_name, dataset_copy, label_encoder.classes_)

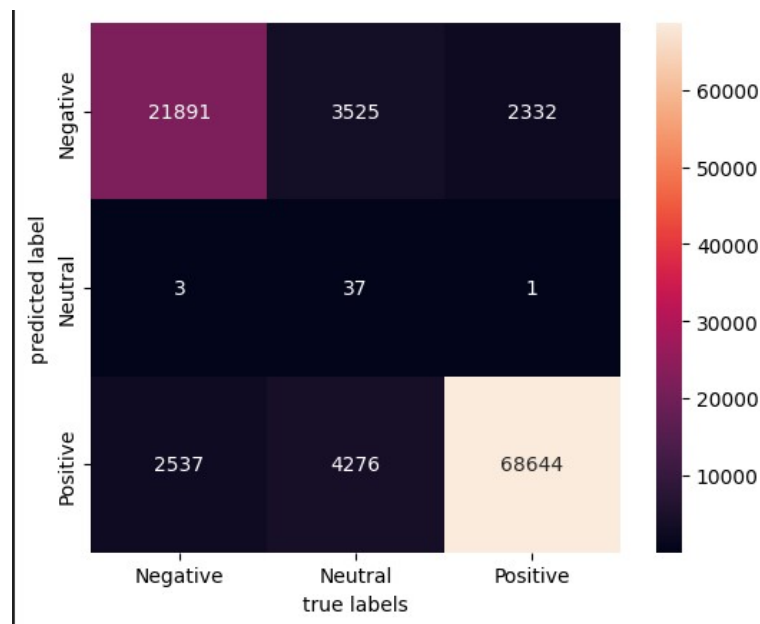
```

```

2420/2420 ————— 301s 123ms/step - accuracy: 0.7036 - loss: 0.7729 - val_accuracy: 0.7285 - val_loss: 0.6956
Epoch 2/3
2420/2420 ————— 298s 123ms/step - accuracy: 0.8401 - loss: 0.4570 - val_accuracy: 0.8699 - val_loss: 0.3639
Epoch 3/3
2420/2420 ————— 299s 124ms/step - accuracy: 0.8757 - loss: 0.3520 - val_accuracy: 0.8772 - val_loss: 0.3372
3227/3227 ————— 52s 16ms/step
Accuracy: 0.8772446390174922
Classification report :

```

	precision	recall	f1-score	support
Negative	0.79	0.90	0.84	24431
Neutral	0.90	0.00	0.01	7838
Positive	0.91	0.97	0.94	70977
accuracy			0.88	103246
macro avg	0.87	0.62	0.60	103246
weighted avg	0.88	0.88	0.84	103246



Primer treniranja i evaluacije veštačke neuronske mreže

Na kraju, svi rezultati su objedinjeni i analizirani radi poređenja performansi svake arhitekture:

```
final_dataframe = pd.concat([dataframe_lstm, dataframe_bilstm,dataframe_gru,dataframe_cnn ], ignore_index=True)
final_dataframe_sorted = final_dataframe.sort_values(by="accuracy", ascending=False)
final_dataframe_sorted
```

✓ 0.0s

	model	accuracy
3	CNN_sentiment_model	0.924443
1	BiLSTM_sentiment_model	0.892674
2	GRU_sentiment_model	0.892664
0	LSTM_sentiment_model	0.877245

Tačnost neuronskih mreža

Rezultati pokazuju da je CNN model postigao najbolju tačnost od oko 92%, dok su BiLSTM i GRU modeli ostvarili slične rezultate od oko 89%. LSTM model je imao nešto slabiji rezultat od 87%. CNN se pokazao najefikasnijim jer najbolje prepoznaje lokalne obrasce u tekstu, dok su BiLSTM i GRU stabilni i pouzdani, ali nešto manje precizni. Ukupno gledano, svi modeli su dali dobre rezultate, ali CNN se izdvojio kao najtačniji za analizu sentimenta.

4. Zaključak

Analiza sentimenta predstavlja jednu od najznačajnijih primena u oblasti obrade prirodnih jezika i dubokog učenja. U savremenom digitalnom dobu, svakodnevno se generiše ogromna količina tekstualnih podataka na društvenim mrežama, forumima, platformama za recenzije i različitim oblicima onlajn komunikacije. Zbog toga se javlja potreba za automatizovanim sistemima koji mogu da prepoznaju emocije, stavove i mišljenja korisnika na osnovu teksta. Upravo tu dolazi do izražaja analiza sentimenta – proces koji omogućava klasifikaciju teksta kao pozitivnog, negativnog ili neutralnog.

Cilj ovog rada bio je da se primeni više različitih pristupa analizi sentimenta i uporede njihove performanse. U okviru projekta korišćeni su kako klasični modeli mašinskog učenja (Logistic Regression, SVM, Random Forest), tako i savremeni modeli dubokog učenja i BERT arhitekture. Klasični pristupi poslužili su kao osnova za razumevanje načina na koji tradicionalne metode rešavaju zadatke klasifikacije teksta, dok su duboke neuronske mreže poput LSTM, BiLSTM, GRU i CNN modela omogućile dublju analizu konteksta i bolje razumevanje odnosa između reči u rečenica.

Tokom izrade projekta stečeno je dublje razumevanje brojnih koncepata koji obuhvataju obradu prirodnih jezika, kao i različitih metoda za preprocesiranje, vizualizaciju, vektorizaciju i evaluaciju modela. Gledajući u budućnost, sa ubrzanim razvojem veštačke inteligencije i dubokog učenja, može se očekivati da će se ovi modeli još više usavršavati i sve češće primenjivati u svakodnevnom životu, čime će dodatno doprineti boljem razumevanju ljudskog jezika, emocija i komunikacije.

5. Reference

- 1) <https://huggingface.co/blog/sentiment-analysis-python>
- 2) <https://brighteshun.medium.com/sentiment-analysis-part-1-finetuning-and-hosting-a-text-classification-model-on-huggingface-gd6da6fd856b>
- 3) <https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/04-n-grams>
- 4) <https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/05-tokenization-stemming-lemmatization>
- 5) <https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/06-classify-articles>
- 6) <https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/09-text-as-vectors-bow-tfidf>
- 7) <https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/13-classify-articles-embeddings>
- 8) <https://huggingface.co/nlptown/bert-base-multilingual-uncased-sentiment>
- 9) <https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest>
- 10) <https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>
- 11) https://www.researchgate.net/publication/378435259_Sentiment_analysis_of_hotel_comments_based_on_LSTM_and_GRU
- 12) <https://www.sciencedirect.com/science/article/pii/S1877050917312103>
- 13) <https://ieeexplore.ieee.org/document/9489129>